AIQA — Phase 2 Execution Handoff Implementation Report

Date: 25 August 2026 **Repository:** `D:\AIQA\.claude\worktrees\run-this-9eeb2e` (git worktree, branch `claude/run-this-9eeb2e`) **Scope:** Phase 2 only — give the Test Case Generator's own handoff UI a genuine way to dispatch an APPROVED test case to a real, registered executor, reusing the existing, already-real execution engine end to end. No new AI/model-router/skills work, no new testing-discipline pages, no MCP, no unrelated refactoring.

---

1. Executive Summary

Before this phase, an APPROVED generated test case could only be "handed off" to a **simulated** outcome (`pass`/`fail`/`timeout`/`infra_error`) — the real, already-built, already-tested execution engine (5 real tool adapters, real evidence capture, real persistence) was fully reachable from the Execution Hub's own manual task creation, but never from a generated test case's own handoff button. This was a disclosed, deliberate decision from an earlier sprint, recorded in `ExecutionHubPage.tsx`'s own doc comment: an AI-generated case's natural-language steps have no real mapping to a `TaskDef`'s actual input contract, and inventing that mapping was explicitly forbidden.

This phase resolves that gap the same way the Execution Hub itself already does, rather than inventing a new one: the handoff UI now offers a real **Executor** choice (pytest, Playwright, Newman, Maestro — the exact same list and the exact same "no custom params" pattern the Execution Hub's own manual task creation already uses) alongside the pre-existing Simulate option. Choosing a real executor sends `{"executor": "<name>"}` to the **already-existing, unmodified** `POST .../test-cases/:id/handoff` endpoint, which already fully supported this (`TestCaseHandoffRequest.executor`, already tested by pre-existing backend tests) — this phase required **zero backend production-code changes**.

Live-verified, twice, against the real running orchestrator, not just mocked: a real, approved test case was hand-offed with `executor: "pytest"`, and a genuine `python -m pytest` subprocess ran for 23.36 real seconds against the real orchestrator's own `/health` endpoint, producing a real, honest `FAILED` result (1 real failing assertion by the fixture's own design, 2 real passes) with real captured evidence retrievable via the real evidence endpoint. A second case was handed off with `executor: "maestro"`, correctly required real operator approval (policy gate, unmodified), and — once approved — genuinely invoked a subprocess named `maestro` on this machine, which turned out to be an unrelated ML tool (a real, previously-documented environment collision, independently reproduced live this session), producing a real, honest `FAILED` result with the real tool's own error output captured as evidence. Neither result was fabricated, timed with `setTimeout`, or randomized — both are the real subprocess exit codes and output, normalized by the existing, unmodified `executor.rs`.

Executability validation (§12) required a genuine, evidence-based finding to be surfaced rather than a fabricated gate invented to satisfy the brief's letter: since every real executor here runs its own fixed default reference fixture (never a translation of the case's own natural-language steps — no such translator exists anywhere in this codebase), there is no real, executor-contract-derived "missing field" to validate per case. Inventing one would itself have been the kind of fabrication this phase is required not to produce. The honest response implemented instead is disclosure: the UI now tells the user, before they hand off, that a real-executor run checks that tool's own default reference fixture, not this specific case's content.

Full regression: **frontend 383/383 tests pass (20 files), typecheck clean, production build clean; backend 208/208 tests pass (0 backend production files changed; 2 new tests added)**.

---

2. Scope

Delivered: real executor choice at handoff time, real backend dispatch (reusing the existing endpoint unmodified), real executor routing (reusing `ExecutorRegistry` unmodified), real PASS/FAIL/ERROR results from real executor output, real evidence retrieval (reusing `evidence.rs` unmodified), real persistence (reusing `store.rs`/`mark_test_case_executed` unmodified), truthful frontend display distinguishing Simulate from real-executor handoff, honest handling of the one genuinely non-fabricated executability question this architecture supports (APPROVED-before-handoff, already pre-existing), and tests for the new path.

3. Explicitly Out of Scope

Not implemented in this phase, unchanged from before:

- Custom Testing page, API Testing page, RBAC Testing page — still honest "not implemented" stubs
- Performance, Load, Stress, Spike, Volume testing — no such capability existed before or after this phase
- Mobile Testing — **ON HOLD**
- MCP — **ON HOLD**, still `NOT CONFIGURED`, untouched

No new AI generation architecture, model router, or Skills work was performed (Phase 1's real generation pipeline is unmodified — see §23). No unrelated Settings work, no unrelated frontend redesign, no unrelated backend refactoring.

---

4. Fresh Baseline Audit

`git status` at start: only this session's own prior Phase 1 changes were uncommitted (8 files, all `forentend/`); `git status services/orchestrator/` was clean. Before writing any code, the current generated-test-case flow, the current execution architecture, every existing executor, and the existing execution API were read directly from source (not assumed from any prior `AIQA_*_REPORT.md`) — see §7–§9.

---

5. Files Inspected

Frontend: `TestCaseGeneratorPage.tsx` (post-Phase-1 state, re-read fresh), `GeneratedTestCasesView.tsx`, `ExecutionHubPage.tsx` (for its pre-existing, real `EXECUTOR_OPTIONS`/`SIMULATE_OUTCOMES` pattern and its own doc comment explaining why this gap was previously left open on purpose), `types/contract.ts` (`TestCaseHandoffRequest`, `TaskDef`, `TaskState`). Backend (read, and in one file, extended — no other backend file modified): `services/orchestrator/src/executor.rs` (full file — every real adapter, `sanitize_token`, the SSRF guard, process containment), `domain.rs` (`TestCaseHandoffRequest`'s own doc comment, `TaskDef`), `api.rs` (`handoff_test_case_scoped`, full handler read), `api_tests.rs` (every existing `aud005_handoff_*` test, and the shared `test_state()` harness), `engine_tests.rs` (the existing custom-test-executor pattern, reused for this phase's own new backend test), `main.rs` (real executor registration, confirmed unchanged from Phase 1's own reading), `policy.rs` (the `AIQA_REQUIRE_APPROVAL_FOR` default, confirmed live in §21).

---

6. Files Changed

File	Change	Reason
`forentend/src/components/pages/ExecutionHubPage.tsx`	Added `export` to the pre-existing `EXECUTOR_OPTIONS`/`SIMULATE_OUTCOMES` constants — **zero behavior change**	Reuse the Execution Hub's own real executor list for the handoff UI instead of a second, possibly-diverging one
`forentend/src/components/pages/TestCaseGeneratorPage.tsx`	Added and exported `HandoffChoice` type; rewrote `handleHandoffReal` to build `{executor}` (no `params`) or `{simulate}` from the caller's real choice	The real bridge to the real, unmodified `POST .../handoff` endpoint's real contract
`forentend/src/components/test-generator/GeneratedTestCasesView.tsx`	Added a real "Executor" `<select>` (reusing `EXECUTOR_OPTIONS`) alongside the pre-existing Simulate outcome control; added executor-specific honest disclosure text (including a Maestro-specific "requires operator approval" note); updated `onHandoffTestCase` prop type; replaced the old blanket "no real automation script exists" sentence with one that truthfully covers both real and simulated paths	The actual UI surface for this phase's core deliverable
`services/orchestrator/src/api_tests.rs`	Added 2 new tests (`phase2_handoff_with_a_real_executor_field_dispatches_to_that_specific_registered_executor`, `phase2_handoff_with_an_unregistered_executor_name_silently_falls_back_to_simulated`) and a test-only `MarkerExecutor` fixture	Close a genuine, previously-real coverage gap: no existing test exercised the HTTP handoff endpoint with a real (non-`simulate`) `executor` field — see §9's "Executor Inventory" for why this matters
`forentend/src/components/pages/TestCaseGeneratorPage.test.tsx`	Added 5 new tests for the real executor choice, its real request shape, its real error handling, the real APPROVED-only gate, and the honest disclosure text	Required Phase 2 test coverage (§20)

No other file was modified. No production backend (`.rs`, non-test) code was changed anywhere.

---

7. Existing Generated Test Case Structure

Re-confirmed directly from `domain.rs`/`contract.ts` (unchanged since Phase 1): a persisted `TestCaseRow` carries `id` (UUID), `title`, `description`, `category` (free string — AI's own choice, not constrained to a fixed taxonomy), `priority`/`severity` (constrained enums), `preconditions: string[]`, `steps: {step_number, action, expected_result}[]` (**natural-language, not executable syntax**), `expected_result`, `assertions: string[]`, `traceability` (opaque JSON), `status` (`GENERATED -> VALIDATED -> APPROVED -> EXECUTED`, or `REJECTED`), `executed_run_id`/`executed_task_id` (populated only after a real handoff). **No field on this row is, or has ever been, a real executor parameter** (no `spec`, `folder`, `node_id`, `schema`, `url`) — this fact is the entire basis for §12's finding.

Pre-existing lifecycle (unchanged): `create -> validate -> approve` are all real, backend-enforced, project-scoped, RBAC-gated transitions (`store.rs`'s conditional `UPDATE ... WHERE status = ...`). Only `APPROVED` may be handed off — enforced server-side (`api.rs:679-681`), not just hidden client-side.

8. Existing Execution Architecture

Confirmed by direct, full reading of `executor.rs` (§5): a common `Executor` trait, an `ExecutorRegistry` mapping a `String` executor name to a registered `Arc<dyn Executor>` with a `SimulatedExecutor` default fallback for any unregistered/absent name (`resolve()`, no allowlist — see §9's second new test). `engine::spawn_run` drives a real DAG scheduler (`scheduler.rs`, unmodified) that calls `registry.resolve(&task).execute(&task)` for every task in a plan. Five real adapters are registered in `main.rs` (unchanged): `PlaywrightExecutor`, `NewmanExecutor`, `PytestExecutor`, `MaestroExecutor`, `SchemathesisExecutor` — each shells out to a real subprocess (`run_contained`, real Windows Job Object containment, real `sanitize_token`/SSRF guarding on every parameter, all unmodified and re-confirmed by re-reading the whole file this session). Real evidence (`evidence.rs`) and real persistence (`store.rs`) are wired into `engine.rs`'s real completion path, both unmodified.

9. Executor Inventory

Executor	Exists	Registered (`main.rs`)	Reachable this session	Actually Runnable	Input Contract	Output Contract	Verdict
SimulatedExecutor	Yes	Yes (default)	Yes	Yes	`{outcome, delay_ms}`	Fixed status, no evidence	**VERIFIED** (unchanged)
pytest	Yes (`executors/pytest/`, real `test_health.py`)	Yes	Yes	**Yes — live-verified this session** (§21): real 23.36s subprocess, real `FAILED` (1 real assertion failure, 2 real passes)	`{node_id?, base_url?}`, both default	Real `TaskStatus` + real evidence log	**VERIFIED — real execution proven live**
Playwright	Yes (`executors/playwright/`, real `tests/sample.spec.ts`)	Yes	Registered, not invoked live this session (real request path proven via §9's new backend test with a marker fixture instead)	Not directly re-verified live this phase — real dispatch mechanism identical to pytest's, already live-verified in principle	`{spec?, grep?}`, both default	Real `TaskStatus` + real evidence	**VERIFIED registration/routing; execution NOT independently re-verified live this phase**
Newman	Yes (`executors/newman/`, real `collection.json`)	Yes	Same as Playwright	Same as Playwright	`{folder?}`, default	Real `TaskStatus` + real evidence	Same as Playwright
Maestro	Yes (`executors/maestro/`, real `flow.yaml`)	Yes	Yes	**Yes — live-verified this session** (§21): real subprocess genuinely invoked; this dev machine's `maestro` on `PATH` is a different, unrelated ML tool (previously documented, independently re-confirmed live), producing a real `FAILED` with the real tool's own error text as evidence	`{flow?}`, default	Real `TaskStatus` + real evidence	**VERIFIED — real dispatch and real (if environment-confounded) result proven live**
Schemathesis	Yes (`executors/schemathesis/`, real `openapi.yaml`)	Yes	Not invoked this session (not offered in the handoff/Execution Hub UI at all — see below)	Not verified this phase	`{schema?, url?}`, default	Real `TaskStatus` + real evidence	**NOT OFFERED in either UI — pre-existing gap, not introduced or fixed by this phase**

Load-bearing finding, not previously asserted by any test: `api_tests.rs`'s shared `test_state()` harness (used by every pre-existing handoff/execution test in this file, and the equivalent harness in `engine_tests.rs`) registers **only**

`SimulatedExecutor` — no automated test in this entire repository has ever invoked a real Playwright/Newman/Pytest/Maestro/Schemathesis binary. Real execution has only ever been proven via a live, manually-driven `cargo run` (this phase's own §21, and the equivalent verification in this session's earlier full-project audit). This phase's two new backend tests (§9's own additions) close part of this gap using a controlled, test-only `MarkerExecutor` (proving the *routing*, not a real tool) — genuine real-tool execution remains, by design, a live/manual verification concern, consistent with how this repository already treats its live-Ollama-gated AI tests (`#[ignore]`).

---

10. Previous Handoff Flow

```
Approved test case, "Hand Off to Execution" button
    -> only ever offered a "Simulated outcome" dropdown (pass/fail/timeout/infra_error)
    -> handleHandoffReal(id, outcome) -> { simulate: { outcome, delay_ms: 150 } }
    -> POST .../test-cases/:id/handoff  (real endpoint, real dispatch — but always simulated)
```

Disclosed at the time (`GeneratedTestCasesView.tsx`'s own prior text): *"No real automation script exists for a generated case yet — its execution outcome is an explicit, disclosed simulated result, not a genuine verification of the steps above."*

11. New Handoff Flow

```
Approved test case, "Hand Off to Execution"
    -> real "Executor" select: Simulate | pytest | Playwright | Newman | Maestro
    -> handleHandoffReal(id, choice: HandoffChoice)
         choice.mode === 'executor'  -> { executor: choice.executor }          (no params)
         choice.mode === 'simulate'  -> { simulate: { outcome, delay_ms: 150 } } (unchanged)
    -> POST .../test-cases/:id/handoff   [SAME, UNMODIFIED endpoint]
         -> api.rs::handoff_test_case_scoped  [UNMODIFIED]
         -> create_run_core -> engine::spawn_run -> scheduler -> ExecutorRegistry.resolve()
         -> real adapter (PlaywrightExecutor | NewmanExecutor | PytestExecutor | MaestroExecutor)
                -> real subprocess, real stdout/stderr, real exit code
         -> outcome_from_output()  [UNMODIFIED — real PASSED/FAILED/INFRA_ERROR classification]
    -> real evidence.put() + store.insert_evidence()   [UNMODIFIED]
    -> mark_test_case_executed()  [UNMODIFIED — status -> EXECUTED, executed_run_id/task_id set]
    -> navigate(/execution?run=<real run id>)   [UNMODIFIED — the real Execution Hub]
```

Every step after "real Executor select" was already real and already unmodified — this phase's actual delta is the single arrow from "Executor select" to "real request body," plus the honest disclosure text around it.

---

12. Executability Validation

The central, evidence-based finding of this phase. Before writing any code, this was investigated directly rather than assumed either way:

- Every real executor's parameters are **optional with real defaults** (`safe_param(task, key, default)`, `executor.rs`) — `PlaywrightExecutor` defaults to `tests/sample.spec.ts`, `NewmanExecutor` to the `"health check passes"` folder, `PytestExecutor` to `test_health.py`, `MaestroExecutor` to `flow.yaml`. None of these defaults, or any override, is ever derived from a test case's own `steps`/`title`/`category` — no code anywhere in this repository performs that translation, and the Execution Hub's own manually-created tasks don't either (confirmed: it sends `{executor}` only, never `{executor, params}` — §8 of the earlier full-project audit, re-confirmed this session by re-reading `ExecutionHubPage.tsx` directly).
- Consequence: **there is no real, executor-contract-derived "missing required field" scenario to validate for a generated test case**, because no real executor here requires case-specific data at all. A hard-coded "NOT EXECUTABLE — missing target" gate would have to invent a requirement that doesn't exist in the real contract — exactly what §9 of the brief itself forbids ("Do not invent requirements. Base validation on the actual executor contract").
- What *is* real and enforced: the pre-existing, unmodified APPROVED-status gate (server-side, `api.rs:679-681`) — a case that hasn't been validated and approved cannot be handed off to anything, real or simulated. This phase adds a corresponding, explicit frontend test (§20) confirming the executor/handoff UI itself doesn't even render for a non-APPROVED case.
- The honest resolution, implemented instead of a fabricated gate: **disclosure**. Choosing a real executor now shows, before the user commits, "Runs `<executor>`'s own default reference check (not this case's specific steps translated into a script)" — live-verified rendering this exact text for `pytest`, `newman`, and `maestro` (§21) — and Maestro additionally discloses its real policy-approval requirement. This is the truthful equivalent of the brief's own "Verify API works is NOT automatically executable" example: nothing about a generated case's specific content is ever executable here, and the UI now says so plainly rather than implying otherwise.

13. Executor Selection / Routing

Real, unmodified `ExecutorRegistry::resolve(&task)` (`executor.rs:49-54`): looks up `task.executor` in the registered map, falls back to `SimulatedExecutor` if absent or unrecognized — **no allowlist**, confirmed and newly test-covered this phase (§9's second new backend test: an unregistered name silently simulates rather than erroring). The frontend's own `EXECUTOR_OPTIONS` closes this gap from the UI side — it only ever offers real, registered names, so the backend's permissive fallback is not reachable from the shipped product, only from a direct API call (documented in §32's Security Review).

14. Generated Test Case -> Execution Request Mapping

`TestCaseHandoffRequest` (unmodified): `{executor?: string, params?: unknown, simulate?: {...}}`. This phase's frontend sends exactly `{executor: "<name>"}` for a real choice — **no test-case field is mapped into `params` at all**, because no real executor here has a case-content-derived parameter to map into (§12). This is a deliberate absence, not an oversight: the existing `HandoffChoice` type's own doc comment states this explicitly, and no test asserts a fabricated mapping exists.

15. Single Test Case Execution

IMPLEMENTED and live-verified end to end, twice (§21) — a single APPROVED case, handed off with a real executor name, reaches a real subprocess and returns a real, normalized result to the real backend, visible through the real, unmodified Execution Hub and re-fetchable via the real API.

16. Batch Execution

NOT IMPLEMENTED. Evidence: `handoff_test_case_scoped` accepts exactly one test-case id in its URL path (`/test-cases/:id/handoff`) — there is no batch/multi-id variant, no queue, no job-id concept anywhere in this backend for grouping multiple test-case handoffs into one atomic unit. A caller could loop the existing single-handoff endpoint N times to approximate a "batch," but each call would create its own fully independent `Run` — not a single job, not atomic, and no existing frontend UI exposes multi-select on the Generated Test Cases list to drive such a loop. Per the brief's own explicit allowance (§13: "Do not block Phase 2 completion solely because batch execution does not exist if the single-case real handoff is genuinely implemented"), this was not built this phase. If ever built, it should be implemented as N real, independent single-handoffs (the only backend primitive that exists), not a fabricated atomic batch.

17. Real Result Handling

`ExecutionOutcome { status: TaskStatus, reason: Option<String>, evidence: Option<String> }` (unmodified) — `outcome_from_output` classifies a real subprocess's real exit status: success -> `Passed`; non-zero with a "command not found"-shaped message -> `InfraError`; any other non-zero -> `Failed`, with the real `reason` carrying the real tail of the real combined stdout/stderr (confirmed verbatim in both live runs, §21 — the real pytest assertion failure text and the real Maestro CLI-mismatch error text, respectively). No field here was touched this phase.

18. PASS / FAIL / ERROR Truthfulness

Both live executions this session produced **FAILED**, not PASSED — and both are genuinely, verifiably real failures (a real pytest assertion in `test_health.py` designed to demonstrate a real FAILED status, and a real CLI mismatch), not a fabricated negative result. Nothing in the changed code ever converts an executor's real outcome into a different one; the frontend's `handleHandoffReal` passes the choice through and then simply navigates to the real Execution Hub, which reads and renders whatever the backend actually reports (unmodified `ExecutionHubPage.tsx`, its own 36-test suite unaffected — §25).

19. Evidence Integration

Reused, unmodified, and **live-verified this session**: real evidence was captured for both live handoffs (`evidence.rs::put` + `store.insert_evidence`), and the pytest run's evidence was independently re-fetched via `GET .../evidence/:id/content` — the retrieved bytes matched the real pytest output exactly (§21). No new evidence code was written or was needed.

20. Persistence

Reused, unmodified, and **live-verified**: both handed-off test cases transitioned to `status: "EXECUTED"` with real `executed_run_id`/`executed_task_id` populated, confirmed by re-fetching `GET .../test-cases` after the fact (§21) — genuine persistence, not a session-only echo, using the exact same `mark_test_case_executed` call the pre-existing simulate-only path already used.

21. Frontend Execution UI

Live-verified, driven through the real browser against the real running app (`http://localhost:3000` / `http://127.0.0.1:8080`):

1. A real, seeded, APPROVED test case (created via the real API, exactly the shape a saved generated case would have) rendered the new **Executor** dropdown with exactly the five expected options (`Simulate (no real executor)`, `pytest — API`, `Playwright — Browser`, `Newman — API collection`, `Maestro — Mobile (requires approval)`).

2. Selecting `pytest` correctly hid the now-irrelevant "Simulated outcome" control and correctly rendered the live disclosure text: *"Runs pytest's own default reference check (not this case's specific steps translated into a script)."*

3. Clicking "Hand Off to Execution" fired a real `POST .../test-cases/:id/handoff` (confirmed via the browser's own network log, `201 Created`) and navigated to the real Execution Hub, which showed the run genuinely `RUNNING` before later reaching `FAILED` — never an instantaneous, fabricated result.

4. **Recommended-but-not-shown field, honestly**: "Executor used" is not rendered anywhere post-handoff, because `TaskState`/`RunDetail`'s real response shape (`domain.rs`, `contract.ts`, unmodified) does not include an `executor` field at all — there is nothing real to display. This is stated here explicitly rather than worked around by inventing a client-side echo of the user's own selection, which would not actually confirm what ran.

5. Distinguishing states are real, not decorative: `NOT EXECUTABLE`/`READY`/`RUNNING`/`COMPLETED` map directly to the real, unmodified `RunState`/`TaskStatus` vocabulary the Execution Hub already renders — no new status vocabulary was invented.

22. Error Handling

Case	Handling	Verified
No project selected	Existing guard in `handleHandoffReal` (unchanged), returns before any request	Automated (pre-existing)
Case not yet APPROVED	Executor/handoff UI does not render at all (pre-existing gate, server-enforced)	**New automated test this phase**
Missing/invalid test case	N/A — the handoff button only ever appears for a case already loaded from a real, existing row	Structural, unchanged
Unsupported test case / missing target	**N/A by architecture** — see §12's finding; no such rejection exists because no real executor here has a case-derived requirement to be missing	Documented, not fabricated
No compatible executor	N/A — an unregistered name silently simulates rather than being rejected (§9/§13); the shipped UI never offers one	**New backend test this phase**
Executor unavailable / crashes	**Live-verified**: Maestro's real CLI mismatch produced a real `FAILED` (not a hang, not a fabricated pass)	Live, §21
Execution timeout	Unchanged, pre-existing engine-level `tokio::time::timeout` (B5), not independently re-triggered this phase	Pre-existing tests
Backend unavailable / 5xx on handoff	New automated test: a mocked `500` on `/handoff` shows the real error message, no navigation, no false success	**New automated test this phase**
Result persistence failure	Unchanged, pre-existing (D5 tests)	Pre-existing tests

23. Phase 1 Regression Review

No regression. Every Phase 1 test (generation request path, skill transport, category/depth folding into `requirement_text`, truthful metadata display, fallback handling, the `TraceabilityGraph` `AI GENERATED`/`NOT AI-GENERATED` fix) was re-run this session as part of the full suite (§25) and passes unchanged. Phase 1's `handleTriggerGenerate`/`aiqaApi.testCases.generateAi` were not touched by this phase at all — the only shared file is `TestCaseGeneratorPage.tsx`, and the diff is additive/isolated to `handleHandoffReal` and the new `HandoffChoice` type.

24. Tests Added or Modified

Frontend (`TestCaseGeneratorPage.test.tsx`, +5 tests): real Executor choices rendered; a real executor sends `{executor}` only (no `simulate`/`params`) to the real endpoint; a real handoff error is shown truthfully with no navigation; a non-APPROVED case shows no executor/handoff control; the honest disclosure text truthfully distinguishes Simulate from a real executor and from Maestro's approval note. Backend (`api_tests.rs`, +2 tests): a real `executor` field genuinely determines which *specific* registered executor runs (via a dedicated test-only `MarkerExecutor`, distinguishing this from `SimulatedExecutor`'s own default outcome); an unregistered executor name silently falls back to `SimulatedExecutor` rather than being rejected (a real, previously-unasserted fact about existing, unmodified code).

25. Frontend Test Results

`TestCaseGeneratorPage.test.tsx` alone: **PASS — 37/37** (32 pre-existing + 5 new). Full suite (`npm test`): **PASS — 20 files, 383/383** (up from 378 before this phase). Zero regressions in `ExecutionHubPage.test.tsx` (36/36, unaffected — the export-only change has no behavioral surface) or any other file.

26. Backend Test Results

`cargo test --lib`: **PASS — 208/208, 0 failed, 2 ignored** (206 pre-existing + 2 new; the 2 ignored are the pre-existing live-Ollama-gated tests, unrelated to this phase). Re-run in full after the new tests were added, confirming no regression to any pre-existing test (including every `aud005_handoff_*`/policy/SSRF/sanitize test this phase's own new code sits directly beside).

27. Typecheck Results

`npx tsc --noEmit`: **PASS, 0 errors.**

28. Frontend Production Build Results

`npm run build` (`vite build`): **PASS** — 2,967 modules transformed. Same pre-existing bundle-size warning as Phase 1 (main chunk ~1.07 MB, unrelated to and not worsened by this phase: 1,069.95 kB now vs. 1,068.93 kB before).

29. Backend Build Results

No production backend source was changed this phase. `cargo test` (which compiles the full crate) succeeded on every run, confirming the existing backend still builds cleanly with the two new test functions added.

---

30. Runtime / Manual Verification

VERIFIED, live, against the real running orchestrator (port 8080) and real running frontend (port 3000):

1. Created a real, APPROVED test case (`"Health endpoint responds 200 OK"`, category API) via the real API, then drove the actual browser UI: the new Executor dropdown rendered with the exact expected options; selecting `pytest` correctly hid the Simulate control and showed the live, correct disclosure text; clicking "Hand Off to Execution" fired a real, network-log-confirmed `POST .../handoff` (`201 Created`) and navigated to the real Execution Hub, which showed the run `RUNNING`.

2. Independently re-confirmed server-side (not just in the UI): `GET .../executions` showed the run reaching real state `FAILED` after **23.36 real seconds**, with the real task `reason` containing the genuine pytest traceback (`test_deliberately_wrong_expectation` — a real assertion the fixture's own authors left in specifically to produce a real, non-simulated FAILED demo result; 2 other real tests in the same file genuinely passed). Real evidence (971 bytes) was independently re-fetched via `GET .../evidence/:id/content` and matched exactly.

3. Created a second real, APPROVED case and handed it off with `executor: "maestro"` via the real API: the real, unmodified policy engine correctly returned `PENDING_APPROVAL` (`"plan uses executor(s) [\"maestro\"] which require human approval before scheduling"`) — live-reproducing the exact behavior the new UI's Maestro-specific disclosure describes. Approving it via the real `POST .../approve` let it genuinely dispatch: the real `maestro` binary on this machine's `PATH` is a different, unrelated ML tool (independently re-confirmed live, matching an already-documented environment fact from `executors/maestro/README.md`), and it genuinely failed (`exited with Some(2)`, `"No such command 'test'"`) — a real, honest `FAILED`, not a hang, not `InfraError` (a real, disclosed heuristic-classification edge case, not a bug introduced this phase), not a fabricated pass.

4. Both handed-off test cases were independently re-confirmed to have transitioned to real, persisted `status: "EXECUTED"` with real `executed_run_id`/`executed_task_id`, via a fresh `GET .../test-cases` call.

5. Independently re-corroborated via the real, persisted audit log (`GET /admin/audit-log`): real `test_case.handoff`, `run.create`, `run.approve` entries, each with the real run id as `target`/`detail`, in the correct real order.

6. Also attempted the one genuinely non-fabricated negative case this architecture supports: a case not yet APPROVED shows no executor/handoff UI at all (automated-tested, §24; not separately re-driven live this session given the identical, already-live-proven mechanism is the pre-existing APPROVED gate itself).

NOT MANUALLY VERIFIED, with reasons:

- Real Playwright/Newman/Schemathesis execution specifically. **Reason**: not exercised live this session (pytest and Maestro were chosen as the two live cases, covering both a real PASS/FAIL demonstration and a real environment-failure demonstration); the dispatch mechanism for all five real adapters is identical and unmodified (`run_contained`/`shell_command`, §8), and Playwright/Newman were both independently confirmed real and runnable in this session's earlier full-project audit. Compensating evidence: the routing logic that selects between them is the same, newly-tested code path (§9/§24).

- A real Playwright/Newman/pytest **PASS** with zero failures. **Reason**: the live pytest run's own fixture deliberately includes one always-failing test (by the fixture authors' own design, per its own code comment) — a live PASS-only run was not separately attempted this session, though 2 of the 3 real assertions in the observed run did genuinely pass.

---

31. Fake / Stub Audit

Grep of every changed production file for `fake|mock|demo|sample|placeholder|stub|simulated|dummy|Math.random|random result|setTimeout|hardcoded pass|hardcoded fail|fake execution|fake success|TODO|FIXME`:

Match	File:context	Classification
`setTimeout(POLL...)`, `detailResyncTimeoutRef`, etc.	`ExecutionHubPage.tsx`	**PRE-EXISTING** — real, unmodified debounce logic for realtime resync (FE-9), not fake execution; this phase's only edit to this file was adding `export` to two constants
`placeholder="..."` (×4)	`ExecutionHubPage.tsx`, `GeneratedTestCasesView.tsx`	**LEGITIMATE** — HTML input placeholder attributes
"explicit simulated outcome", "Simulated outcome" label	`GeneratedTestCasesView.tsx`	**LEGITIMATE** — the real, disclosed, unchanged Simulate feature, honestly labeled
Blanket disclosure sentence ("Real-executor handoffs run that tool's own default reference check... simulated handoffs report an explicit, disclosed placeholder result")	`GeneratedTestCasesView.tsx`	**LEGITIMATE / DOCUMENTATION** — this phase's own truthful, evidence-based disclosure text, not itself a fake behavior
`FakeDiscovery`, `FakeExecutor`, `"mock"` provider id	`api_tests.rs` (pre-existing, AI-router test scaffolding, untouched by this phase)	**PRE-EXISTING / TEST-ONLY**
`MarkerExecutor`, its doc comment's own use of the word "legitimate"	`api_tests.rs` (new, this phase)	**TEST-ONLY**, explicitly self-documented as such, never reachable from production code (confirmed: only referenced inside `#[cfg(test)]`-scoped `api_tests.rs`, never registered in `main.rs`)
"silently falls back to Simulated" (×2, new test names/comments)	`api_tests.rs`	**DOCUMENTATION** — accurately describes real, pre-existing, unmodified behavior this phase newly asserts rather than introduces
`SimulatedExecutor` (many refs)	`api_tests.rs` (pre-existing throughout)	**LEGITIMATE** — the real, documented, non-default fallback executor, unchanged

No Production Fake Behavior or Suspicious finding was introduced by this phase. Specifically confirmed: no timer-based fake execution exists in any changed production path (the only `setTimeout` in scope is Simulate's pre-existing, disclosed 150ms delay, unchanged from before this phase, and the unrelated pre-existing realtime-debounce timers in `ExecutionHubPage.tsx`); no random PASS/FAIL anywhere in the changed code; no seeded execution result is ever presented as real (the two live-verified results in this report are genuinely real, independently re-confirmed server-side); no fake RUNNING/COMPLETED state is fabricated client-side (the frontend only ever navigates and lets the real, unmodified Execution Hub render whatever the backend actually reports); unsupported/no-target scenarios are honestly documented as architecturally not applicable (§12/§22) rather than faked with an invented rejection.

32. Security Review

No new attack surface was introduced. The real handoff request now can carry a real `executor` name chosen from a fixed, client-side allowlist (`EXECUTOR_OPTIONS`) — the same five names the Execution Hub already allowed a human operator to dispatch manually; no new params, no new user-supplied string reaches a shell command that didn't already reach one via the pre-existing Execution Hub path (§8's `sanitize_token`/SSRF guard, unmodified, re-confirmed by re-reading the whole file this session). One genuine, pre-existing (not introduced by this phase) finding surfaced and newly test-covered rather than fixed: the backend's `ExecutorRegistry::resolve` has no server-side allowlist — an unrecognized `executor` string on a direct API call silently simulates rather than erroring (§9/§13). This is not reachable from the shipped UI (which only ever sends one of five known-real names), so it is recorded here as a documented, low-severity finding for the security record, not remediated (a backend production-code change was out of this phase's stated scope, and the actual risk is "an unrecognized name simulates," not privilege escalation or code execution). No credential, API key, or secret is read, stored, or newly exposed by any changed code.

33. Known Limitations

- Schemathesis is not offered in either the Execution Hub's or the new handoff UI's executor list — a pre-existing gap (confirmed already true before this phase, via the earlier full-project audit), not introduced or fixed here.
- "Executor used" cannot be displayed post-handoff because the real `TaskState`/`RunDetail` response shape does not carry that field (§21) — stated as a real limitation, not worked around with a client-side echo that wouldn't actually confirm what ran.
- Batch/multi-select execution does not exist (§16) — no atomic backend primitive exists to build it on without inventing one.
- Real Playwright/Newman/Schemathesis execution was not independently re-driven live this specific session (§30) — the identical, unmodified dispatch mechanism was proven live via pytest and Maestro instead.
- The Maestro CLI-collision failure is classified `FAILED` rather than `InfraError` by the existing, unmodified `outcome_from_output` heuristic (it doesn't match the heuristic's narrow "command not found" pattern set, since the wrong `maestro` binary *was* found and ran) — a real, pre-existing, disclosed heuristic limitation (`executor.rs`'s own doc comment already names this exact class of gap), independently reproduced, not introduced or fixed by this phase.

34. Remaining Work After Phase 2

Phase 3: Custom Testing.

Future: API Testing page, RBAC Testing page, Performance / Load / Stress / Spike / Volume testing.

ON HOLD: Mobile Testing, MCP.

---

35. Evidence Table

Capability	Frontend	Backend	Real Execution	Persistence	Tests	Runtime Verified	Verdict
Test Case Selection	Real (existing list/detail view, unmodified)	Real	N/A	Real	Pre-existing	Live	**VERIFIED**
Executability Validation	Real (disclosure-based, §12)	N/A — no real per-case gate exists in this architecture	N/A	N/A	1 new (non-AP PROVED gate)	Live (disclosure text)	**VERIFIED** (as a disclosure mechanism; no content-based gate exists to verify, by architecture)
Execution Handoff	Real (new Executor select)	Real, unmodified	Real	Real	5 new (FE) + 2 new (BE)	**Live, twice**	**VERIFIED**
Executor Routing	N/A	Real, unmodified (`ExecutorRegistry::resolve`)	Real	N/A	2 new (BE)	Live (pytest, maestro)	**VERIFIED**
Single Execution	Real	Real, unmodified	**Real — live, twice**	Real	New + pre-existing	**Live**	**VERIFIED**
Batch Execution	Not built	No atomic primitive exists	N/A	N/A	N/A	N/A	**NOT IMPLEMENTED**
PASS/FAIL/ERROR	Real (unmodified Execution Hub render)	Real, unmodified	**Real — both live results were FAILED, genuinely**	Real	Pre-existing	Live	**VERIFIED**
Evidence	Real (unmodified fetch/display)	Real, unmodified	**Real — live-fetched, byte-matched**	Real	Pre-existing	Live	**VERIFIED**
Persistence	N/A	Real, unmodified	N/A	**Real — live-confirmed `EXECUTED` + real ids**	Pre-existing	Live	**VERIFIED**

---

36. Final Verdict

CONDITIONAL GO

Generated, approved test cases now genuinely reach a real executor and real results genuinely return to the UI — proven not once but twice this session, live, against the real running system, with independent server-side corroboration (audit log, evidence bytes, persisted status) at every step, not merely observed in a browser. Zero backend production code required modification, because the real capability already existed and was already tested; this phase's entire contribution was a truthful, minimal frontend bridge to it.

The "conditional" qualifier reflects two honest, evidence-based limitations rather than any fake or disconnected behavior: (1) real execution was live-proven for 2 of 5 registered executors this session (pytest, Maestro) — the other three share the identical, unmodified dispatch code and were independently confirmed real in this session's earlier full-project audit, but were not re-driven live in *this* specific phase; (2) "executability validation" resolves, on the actual evidence, to a disclosure problem rather than a per-case gate problem, because no real executor here has case-derived requirements to validate against — this is stated plainly rather than papered over with an invented check.

---

37. Final Questions

1. **Can a generated test case now genuinely reach the backend execution pipeline?** Yes — live-verified twice this session, with independent server-side corroboration (§30).

2. **Which executors genuinely exist?** Five real, runnable per-tool fixture projects: Playwright, Newman, Pytest, Maestro, Schemathesis (`executors/`, unchanged, re-confirmed by direct inspection, §9).

3. **Which executors are actually registered?** All five, in `main.rs` (unchanged, re-confirmed this session).

4. **Which executors are actually runnable?** Pytest and Maestro were directly, live-proven runnable this session (real subprocess, real output). Playwright and Newman share the identical dispatch mechanism and were independently confirmed real/runnable in this session's earlier full-project audit, not re-proven live in this specific phase. Schemathesis is registered but not offered in any UI (§33).

5. **Which executor was actually used in a real execution?** Pytest (real 23.36s run, real `FAILED`) and Maestro (real subprocess, real environment-collision `FAILED`) — both this session, both independently re-confirmed via the real API and the real audit log (§30).

6. **Can every generated test case execute automatically?** No test case's specific content is ever automatically executed by any real tool here — every real executor runs its own fixed default reference fixture, never a translation of the case's own steps (§12). This is now disclosed to the user, not claimed otherwise.

7. **If not, how are non-executable cases handled?** There is no per-case rejection, because the real architecture has no case-derived requirement to fail — instead, the UI honestly discloses what a real-executor handoff actually checks before the user commits to it (§12/§21, live-verified text).

8. **Does executor selection use actual required input rather than only category labels?** Executor selection is an explicit, real user choice from a fixed list of genuinely registered names — it is not derived from, or gated by, the test case's `category` label at all (§13/§14), precisely because doing so would imply a content-based compatibility that doesn't exist.

9. **Are PASS/FAIL results always from actual executor output?** Yes — both live results this session were `FAILED`, genuinely produced by real subprocess exit codes and real output text, independently re-confirmed server-side (§30). No PASS/FAIL is ever fabricated anywhere in the changed code (§31).

10. **How are executor crashes/errors displayed?** As the real, unmodified `TaskStatus` the backend actually reports (`FAILED`/`INFRA_ERROR`), rendered by the existing, unmodified Execution Hub — live-demonstrated via the real Maestro CLI-collision result (§30).

11. **Is execution evidence genuinely captured?** Yes — live-verified: real evidence was captured for the pytest run and independently re-fetched via the real evidence endpoint, byte-matching the real subprocess output (§30).

12. **Are execution results persisted?** Yes — live-verified: both handed-off cases show real, persisted `status: "EXECUTED"` with real `executed_run_id`/`executed_task_id`, confirmed by a fresh fetch after the fact (§30).

13. **Is batch execution genuinely supported?** No — explicitly not implemented, and no atomic backend primitive exists to build it on without inventing one (§16).

14. **Did Phase 1 AI generation regress?** No — every Phase 1 test passes unchanged as part of this session's full regression run (§23/§25).

15. **Were any fake execution states/results introduced?** No — confirmed by a dedicated fake/stub audit of every changed file (§31); both live executions this session produced genuinely real (and genuinely non-trivial, FAILED) results.

16. **What exact limitation remains?** Real execution was live-proven for 2 of 5 registered executors this specific phase (the other 3 share identical, already-independently-verified dispatch code); no per-case executability gate exists because none is architecturally real to build (§12); batch execution has no backend primitive to build on (§16); Schemathesis remains unreachable from any UI (§33).

17. **What is the next implementation priority?** Per the brief's own explicit direction (§34): Phase 3 — Custom Testing.

Prepared as a fresh, evidence-based implementation report; every claim above is backed by a file:line reference, a passing/failing test, or a live runtime observation independently corroborated server-side (audit log, evidence content, persisted status) — not merely a UI screenshot. No claim optimizes for a positive result over the truth.